
UNDERSTANDING OF LONG TEXTS

Girgorii Vinogradov
vinoggri@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic

ABSTRACT

Long-text embedding requires compressing documents that exceed the context window of standard transformer encoders into fixed-size embeddings while preserving both global topic information and rare but discriminative details. In this work we study memory-based pooling as an alternative to naive chunk aggregation. We first introduce a nonparametric memory mechanism that processes a sequence of chunk representations left-to-right and compresses them into a small set of memory slots using cosine-similarity routing, online (count-based) slot updates, and slot-level pooling. We evaluate two variants: (i) a static memory with fixed capacity and forced replacement when novel content is encountered, and (ii) a dynamic memory that allocates additional slots for novel representations until a maximum capacity is reached. Experiments on WikimQA, NarrativeQA, and QMSum with Qwen3-Embedding and XLM-RoBERTa show that nonparametric memory is competitive with a chunking + mean pooling baseline and can provide modest improvements depending on the dataset and backbone model, with the strongest gains observed for Qwen3-Embedding. Finally, we propose a parametric extension in which memory writing is implemented by an attention-gated GRU and memory pooling by learnable attention; however, training this module was unstable in the current setting. Overall, the results indicate that simple nonparametric memory provides a practical and robust approach to long-text embedding, while fully learnable memory requires additional stabilization and objective design to be reliably effective for text embedding.

1 Introduction

In recent years, the solutions to text classification, ranking, and retrieval problems in the field of NLP have changed significantly due to the development of language models, including LLM. We observe a key change in the method of text embedding, which is the transformation of text into a vector numerical representation, which encapsulates the semantic meaning of the text. New language models based on transformer architecture, which use different variations of the attention mechanism, show excellent results in tasks where it is important to understand not only which words appear in the text and how important these words are, but also the overall context of the text. This approach allows for high-quality text embeddings, encoding more accurate, context-rich information into the final embedding vector.

Unfortunately, such models are not a solution to all problems. More precisely, working with long texts using such models is difficult and represents a non-trivial task. In this work, we will examine this issue and try to draw some useful and reasonable conclusions about working with long texts in terms of understanding them through the creation of embedding vectors.

2 Related work

As mentioned above, working with long texts and, more specifically, creating good, high-quality embeddings is a non-trivial task. The main problem is that LLMs have a limited context window, which means that they can only embed a limited sequence of tokens. For example, BERT [1] and XLM-RoBERTa [2] models, which are encoder-only models, have a context window of 512 tokens. This means that texts with a tokenized representation exceeding 512 tokens must be truncated to exactly this length. There is some widely useful approaches on how to ensure that the whole document is taken into account. In [3], authors experiment with splitting the long text into chunks and then process each chunk

independently, and then aggregating the metric, which in this case the probability of the class for the whole document. This type of approach does not necessarily require any fine-tuning or new layer training. Other approach for chunking is to make the chunking not fixed length, but make the chunking dynamic, this involves designing some mechanism, which will decide where to draw chunk borders [4, 5, 6].

Another approach for aggregating the chunk embeddings is to design a layer that aggregates the text chunks. One of approaches to it is the hierarchical attention approach. This approach includes two stages of attention. Firstly, we split the texts into chunks (in [7] the chunking was performed based on sentences) and then passed them to the word (token) wise attention layers, which contextualize the first level of the text hierarchy. Then, in the second stage, the word-wise embeddings are passed to the transformer-like layer with the attention mechanism to contextualize the sentences. [7, 8]

Generally, all above work tries to make the final model to include all context and all information from the text to the final embedding, regardless of the model's context length. In [9, 10, 11] several methods have been used to include (memorise) the context information about the previous segment context in the current segment processing. This was applied to recursive transformers (MeSH [11]) and to the non-recursive attention transformers (Titans and Transformer-XL models [9, 10])

3 Nonparametric memory

In this section we introduce a nonparametric memory mechanism for building long-text embeddings from transformer hidden states. The main goal is to retain information from long sequences without training an additional memory network, relying instead on deterministic similarity-based routing and simple statistics. The approach operates at the level of chunk representations: given a long input, the base transformer is applied chunk-by-chunk, and each chunk is converted into a single vector by either EOS pooling (taking the last valid token representation) or masked mean pooling over tokens within the chunk. Since different texts may produce a different number of chunks, chunk vectors are padded to a common length and accompanied by a corresponding attention mask.

3.1 Static memory

The static memory variant implements a lightweight, nonparametric analogue of recurrent external memory writing. Conceptually, it follows the same high-level pattern as memory-augmented sequence models such as Titans [9]: the input is processed from left to right, and each new representation is either merged into an existing memory state or stored as a new representative state. However, unlike Titans-style modules that typically employ learnable controllers (e.g., learned projections, gating, or GRU-like updates), our implementation removes the parametric controller entirely and replaces it with deterministic, similarity-based routing and online averaging. This design choice keeps the memory mechanism train-free, while still retaining the key inductive bias of iterative memory accumulation over a long sequence.

The processing of the input text is begin with the fixed-size ($\mathbf{T}$ tokens) text chunking into $\mathbf{C}$ segments and the forward pass of the base transformer model. The result of this part is last hidden state $h \in \mathbb{R}^{\mathbf{C} \times \mathbf{T} \times \mathbf{D}}$ (each hidden state vector of size $\mathbf{D}$). Next, we do a chunk-level pooling:

$$h_{pool} \in \mathbb{R}^{\mathbf{C} \times \mathbf{D}} = \text{Pool}(h)$$

Consequently, we initialize a memory $\mathcal{M} \in \mathbb{R}^{\mathbf{M} \times \mathbf{D}} = \{m_1, m_2, \dots, m_{\mathbf{M}}\}$ of size $\mathbf{M}$ by sampling without replacement the pooled hidden state h_{pool} . If the memory size is less than chunk number $\mathbf{C}$, then the memory vector is simply repeated to fill all $\mathbf{M}$ slots. Also, we initialize the vector of write counts $C = \{c_1, c_2, \dots, c_{\mathbf{M}}\}$ with ones.

Memory writing then proceeds sequentially over the chunk positions $t = 1, \dots, \mathbf{C}$. For each chunk embedding x_t , we compute cosine similarity to each slot:

$$s_{t,j} = \cos(x_t, m_j).$$

The most similar slot is selected by:

$$j^* = \arg \max_j s_{t,j}.$$

We then decide whether to merge x_t into an existing slot m_{j^*} or to trigger a forced rewrite. In the merge case, we update the chosen slot using an online running-mean update with step size determined by the slot's write count:

$$\alpha = \frac{1}{c_{j^*}}, \quad m_{j^*} \leftarrow (1 - \alpha)m_{j^*} + \alpha x_t, \quad c_{j^*} \leftarrow c_{j^*} + 1.$$

If no memory slot is sufficiently similar to the current chunk embedding, the algorithm assumes that x_t represents information not captured by the current set of prototypes. In that case, a forced rewrite is performed: the least occupied slot is selected,

$$j_{min} = \arg \min_j s_{t,j},$$

and overwritten with the new chunk embedding:

$$m_{j_{min}} \leftarrow x_t, \quad c_{j_{min}} \leftarrow 1.$$

Finally, after processing all chunks, the memory is converted into a single document embedding via count-weighted pooling.

$$w_j = \frac{c_j}{\sum_{k=1}^M c_k}, \quad e = \text{normalize}\left(\sum_{j=1}^M w_j \text{normalize}(m_j)\right).$$

The resulting e is the embedding of the whole text.

3.2 Dynamic Memory

The idea behind the Dynamic memory approach is not to force the memory to be exactly of size M (however, we restrict the memory to be more than M). Initialization is deliberately minimal. Instead of sampling vectors to populate a fixed memory, the dynamic memory initializes exactly one slot per document using the first valid chunk embedding:

$$m_1 \leftarrow x_1, \quad c_1 \leftarrow 1.$$

Writing proceeds sequentially over chunk positions. For each chunk vector x_t , routing is performed only over the currently allocated slots:

$$s_{t,j} = \cos(x_t, m_j), \quad j^* = \arg \max_j s_{t,j}, \quad j_{min} = \arg \min_j s_{t,j}.$$

Then, the algorithm decides if new memory should be allocated (similarly to the static memory approach, but no rewriting).

```

if  $\forall t : s_t < \theta$  then
  if  $n = M$  then
     $j_{min} \leftarrow \arg \min_{1 \leq j \leq n} c_j;$ 
     $m_{j_{min}} \leftarrow x_t;$ 
     $c_{j_{min}} \leftarrow 1;$ 
  else
     $n \leftarrow n + 1;$ 
     $m_n \leftarrow x_t;$ 
     $c_n \leftarrow 1;$ 
else
   $\alpha \leftarrow \frac{1}{c_{j^*}};$ 
   $m_{j^*} \leftarrow (1 - \alpha) m_{j^*} + \alpha x_t;$ 
   $c_{j^*} \leftarrow c_{j^*} + 1;$ 

```

The final embedding is computed similarly as in static memory:

$$w_j = \frac{c_j}{\sum_{k=1}^M c_k}, \quad e = \text{normalize}\left(\sum_{j=1}^M w_j \text{normalize}(m_j)\right).$$

The resulting e is the embedding of the whole text.

	Qwen3 - Embedding 0.6B			XLM-RoBERTa-Large (0.6B)		
	WikimQA	NarrativeQA	QMSum	WikimQA	NarrativeQA	QMSum
Chunking + mean pooling	76.85	61.06	51.10	12.55	2.37	13.67
Dynamic Memory	75.68	61.49	50.94	12.61	2.37	13.69
Static Memory	77.87	61.35	51.33	11.65	2.37	13.49

Table 1: nDCG@10 (%) on WikimQA, NarrativeQA, and QMSum. We compare the chunking baseline (mean pooling) against nonparametric Static and Dynamic Memory pooling for Qwen3 and XLM-RoBERTa. Static Memory is best for Qwen3 on WikimQA and QMSum, while Dynamic Memory is best on NarrativeQA and gives small gains for XLM-RoBERTa on WikimQA and QMSum.

4 Evaluation

The evaluation was performed on the LongEmbed dataset [12], specifically on WikimQA, NarrativeQA and QMSum tasks.

The hyperparameters for the evaluation are set accordingly:

- `chunk size = 512`
- `memory size = 10` (for both memory types)
- `Similarity threshold = 0.4` for dynamic memory and 0 for static memory

5 Parametric Memory

In addition to the nonparametric memory mechanisms described previously, we also experimented with a fully learnable (parametric) memory module integrated on top of a transformer encoder. The inspiration was mainly drawn from [9]. The motivation was to replace rule-based memory updates (similarity routing and running means) with an end-to-end trainable memory controller, which could in principle learn what to store, where to store it, and how to aggregate stored information into a retrieval embedding.

In practice, training this parametric memory was unstable and did not learned anything (most likely due to the trainig setup and data). Although this part of the work did not yield a successful trained model, the architecture itself is a reasonable starting point and can be refined in future work by improving optimization stability, scheduling, and regularization. In the following, we describe the proposed parametric design and its intended data flow.

5.1 Memory Module

The parametric memory follows the same high-level interface as the nonparametric memory: it receives a stream of hidden states and incrementally updates a bank of memory cells. Let $h_t \in \mathbb{R}^d$ denote the transformer hidden state at position t , and let $M_t \in \mathbb{R}^{N \times d}$ denote the memory matrix with N cells. The model processes a long input by chunking it, encoding each chunk with the base transformer, and then writing the resulting token-level hidden states into memory sequentially.

Writer: attention-gated GRU update. The core learnable component is a writer that updates all memory cells in parallel at each time step. The writer first computes a single-head attention distribution over memory cells conditioned on the current input h_t , and then applies a GRU-style update to each cell. Concretely, keys and queries are computed as

$$K = W_k M_t \in \mathbb{R}^{B \times N \times d}, \quad q = W_q h_t \in \mathbb{R}^{B \times d},$$

followed by LayerNorm on K and q to stabilize dot products and mitigate collapse. Scaled dot-product attention yields update weights

$$a = \text{softmax}\left(\frac{Kq}{\sqrt{d}}\right) \in \mathbb{R}^{B \times N}.$$

These weights are used as a multiplicative gate on the GRU update, so that only cells receiving high attention are strongly modified. The GRU-style update is defined using reset and update gates

$$r = \sigma(W_r h_t + U_r M_t), \quad z = \sigma(W_z h_t + U_z M_t),$$

and a candidate memory update

$$\widehat{M}_t = \tanh(W_c h_t + U_c(r \odot M_t)).$$

Finally, the memory is updated as

$$M_{t+1} = (z \odot a) \odot \widehat{M}_t + (1 - (z \odot a)) \odot M_t,$$

Learnable memory pooling. After processing the entire sequence, the model converts the final memory matrix into a single embedding vector using a learnable single-query attention pooling module. A trainable query vector $q_p \in \mathbb{R}^d$ attends to the memory cells:

$$\alpha = \text{softmax}\left(\frac{(W_k M_T)(W_q q_p)}{\sqrt{d}}\right) \in \mathbb{R}^{B \times N}, \quad e = \sum_{j=1}^N \alpha_j m_j.$$

The pooled representation is then passed through a feed-forward network and LayerNorm to improve isotropy and stabilize retrieval embeddings. In addition, LayerNorm is applied to token hidden states before writing to memory, as anisotropy in transformer embeddings (especially for RoBERTa-style models) can otherwise dominate similarity computations and attention logits.

The proposed module may be learned on top of any pre-trained transformer, which is able to output the hidden state of the sequence.

6 Conclusion

This work explored memory-based pooling strategies for long-text retrieval embeddings, focusing on approaches that operate on transformer hidden states produced under a chunking pipeline. We first introduced a nonparametric memory mechanism that incrementally compresses a sequence of chunk embeddings into a small set of memory slots using cosine-based routing and simple online updates. Two variants were evaluated: a *static* memory with fixed capacity and forced overwriting for novel content, and a *dynamic* memory that allocates new slots when incoming representations are sufficiently dissimilar to the currently allocated prototypes. The resulting document embedding is obtained via slot-level pooling, where count-based aggregation provides a stable baseline.

Empirically, the nonparametric memory variants were competitive with the chunking baseline and occasionally improved performance depending on the backbone and dataset. In particular, for Qwen3-Embedding the static memory achieved the best results on WikimQA and QMSum, while the dynamic memory slightly improved NarrativeQA. For XLM-RoBERTa, the effects were smaller, with only marginal gains on some datasets. These results suggest that simple nonparametric memory writing can preserve useful structure beyond naive pooling, but the benefit is sensitive to the encoder and the distribution of the retrieval benchmark.

Finally, we attempted to construct a fully learnable parametric memory module, where memory writing is implemented by an attention-gated GRU update and memory pooling is performed by a learnable attention mechanism. While the architecture provides a natural end-to-end generalization of the nonparametric design, training was not successful in the current setting, likely due to optimization instability and degenerate memory behaviors (e.g., collapse or overly sharp attention). Overall, the study indicates that nonparametric memory is a practical and stable direction for long-text embedding under limited training budget, while parametric memory remains promising but requires additional work on training objectives, regularization, and stabilization to become reliably effective for retrieval.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [2] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. *CoRR*, abs/1911.02116, 2019.
- [3] Mohit Tuteja and Daniel González Juclà. Long text classification using transformers with paragraph selection strategies. In Daniel Preotjiuc-Pietro, Catalina Goanta, Ilias Chalkidis, Leslie Barrett, Gerasimos Spanakis, and Nikolaos Aletras, editors, *Proceedings of the Natural Language Processing Workshop 2023*, pages 17–24, Singapore, December 2023. Association for Computational Linguistics.

- [4] Sukjun Hwang, Brandon Wang, and Albert Gu. Dynamic chunking for end-to-end hierarchical sequence modeling, 2025.
- [5] Boheng Sheng, Jiacheng Yao, Meicong Zhang, and Guoxiu He. Dynamic chunking and selection for reading comprehension of ultra-long context in large language models, 2025.
- [6] Hongyu Gong, Yelong Shen, Dian Yu, Jianshu Chen, and Dong Yu. Recurrent chunking mechanisms for long-text machine reading comprehension, 2020.
- [7] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In Kevin Knight, Ani Nenkova, and Owen Rambow, editors, *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489, San Diego, California, June 2016. Association for Computational Linguistics.
- [8] Ilias Chalkidis, Xiang Dai, Manos Fergadiotis, Prodromos Malakasiotis, and Desmond Elliott. An exploration of hierarchical attention transformers for efficient long document classification, 2022.
- [9] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, 2024.
- [10] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019.
- [11] Chengting Yu, Xiaobo Shu, Yadao Wang, Yizhen Zhang, Haoyi Wu, Jiaang Li, Rujiao Long, Ziheng Chen, Yuchi Xu, Wenbo Su, and Bo Zheng. Mesh: Memory-as-state-highways for recursive transformers, 2025.
- [12] Dawei Zhu, Liang Wang, Nan Yang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. Longembed: Extending embedding models for long context retrieval. *arXiv preprint arXiv:2404.12096*, 2024.